

Multimodal Table Scanning Application

Multimodal Table Scanning Application

1. Concept Introduction
 - 1.1 What is "Multimodal Table Scanning"?
 - 1.2 Implementation Principles Overview
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largetmodel/utis/tools_manager.py`)
 2. Model Interface Layer (`largetmodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is "Multimodal Table Scanning"?

Multimodal Table Scanning is a technology that uses image processing and artificial intelligence techniques to recognize and extract table information from images or PDF documents. It not only focuses on visual table structure recognition but also combines multimodal data such as text content and layout information to enhance the understanding of tables. **Large Language Models (LLM)** provide powerful semantic analysis capabilities in understanding this extracted information. Together, they complement each other and jointly improve the intelligence level of document processing.

1.2 Implementation Principles Overview

1. Table Detection and Content Recognition

- Computer vision technology is used to locate tables in documents and OCR technology converts text within tables into editable formats.
- Deep learning methods are used to parse table structures (row/column division, merged cells, etc.) and generate structured data representations.

2. Multimodal Fusion

- Visual (such as table layout), text (OCR results), and possible metadata (such as file type, source) are integrated to form a comprehensive data view.
- Specially designed multimodal models (such as LayoutLM) are used to process these different types of data simultaneously to more accurately understand table content and its contextual relationships.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `scan_table` function in this file defines the execution flow of this tool, especially how it constructs a Prompt requiring Markdown format return.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan table content from image and save to MD file.

        :param args: Arguments containing image path.
        :return: Dictionary with file path and table content.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args: {args}")
        try:
            # Get parameters
            image_path = args.get("image_path") if isinstance(args, dict) else None

            # Smart path fallback mechanism
            if not image_path or not os.path.exists(os.path.expanduser(image_path)):
                self.node.get_logger().warn(f" Image path not specified or invalid
                ('{image_path}').")
                default_path = os.path.join(self.node.pkg_path, "resources_file",
                "scan_table", "test_table.jpg")
                if os.path.exists(default_path):
                    image_path = default_path
                    self.node.get_logger().info(f" Fallback to default test table image for
                    scanning: {image_path}")
                else:
                    error_msg = f"Image path not specified or invalid, and default test
                    table image does not exist at: {default_path}"
                    self.node.get_logger().error(f" {error_msg}")
                    return {"file_path": None, "table_content": None, "error": error_msg}
            else:
                # Resolve the ~ symbol to the actual path
                image_path = os.path.expanduser(image_path)

            # Construct a prompt asking the large model to convert the table in the image to
            Markdown format
            if self.node.language == 'zh':
                prompt = "Please carefully analyze this image and convert the table content
                into Markdown format completely. Ensure the format is correct and do not include any other
                irrelevant text."
            else:
```

```

        prompt = "Please carefully analyze this image and convert the table content
into Markdown format. Ensure the format is correct and do not include any other irrelevant
text."

        # Use the current platform's multimodal capabilities for table recognition
        self.node.get_logger().info(f"Using {self.node.model_client.llm_platform}
platform for table scanning")

        # Use the generic multimodal inference interface, supporting all platforms
        try:
            # Call the generic image inference interface
            # Use an independent message context to avoid influence from historical
image analysis
            message_to_use = [{
                "role": "system",
                "content": "You are a table scanning assistant. Your task is to analyze
images and convert table content into Markdown format. Provide only the Markdown table
content without any additional text."
            }]
            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # Process the result - ensure it is a string type
            if isinstance(result, dict):
                response_text = result.get('response', '')
            else:
                response_text = str(result)

            # Ensure response_text is a string
            if not isinstance(response_text, str):
                response_text = str(response_text)

        except Exception as e:
            self.node.get_logger().error(f"Failed to analyze table with
{self.node.model_client.llm_platform}: {e}")
            return

        # Save the result as an MD file
        image_dir = os.path.dirname(image_path)
        image_filename = os.path.basename(image_path)
        image_name_without_ext = os.path.splitext(image_filename)[0]
        md_filename = f"{image_name_without_ext}_table.md"
        md_file_path = os.path.join(image_dir, md_filename)

        with open(md_file_path, 'w', encoding='utf-8') as f:
            f.write(f"# Scanned Table Result\n\n")
            f.write(response_text)

        self.node.get_logger().info(f"Table scan result saved to: {md_file_path}")

        # Return structured data containing the file path and table content
        return {
            "file_path": md_file_path,

```

```

        "table_content": response_text
    }

except Exception as e:
    self.node.get_logger().error(f"Failed to execute scan_table tool: {e}")
    return None

```

2. Model Interface Layer (`largemodel/utis/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```

# From largemodel/utis/large_model_interface.py

class model_interface:
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface"""
        self.messages = message if message is not None else self.messages
        prompt = text if text else self.system_text.get('text1', "Describe this image.")
        self.messages.append({"role": "user", "content": prompt})

        if not self.client:
            return {'response': f"Client for platform {self.llm_platform} is not initialized.", 'messages': self.messages}

        try:
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, image_path=image_path)
            elif self.llm_platform in ['tongyi', 'qianfan', 'openrouter']:
                response_content = self.openai_compatible_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'spark':
                response_content = self.spark_infer(self.messages, image_path=image_path)
            else:
                response_content = f"Image inference not supported for platform:
{self.llm_platform}"
        except Exception as e:
            response_content = f"Image inference error on platform {self.llm_platform}: {e}"

        self.messages.append({"role": "assistant", "content": response_content})
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

The table scanning function is a typical application of converting unstructured image data into structured text data. Its core technology is still **guiding model behavior through Prompt Engineering**.

1. Tool Layer (`tools_manager.py`):

- The `scan_table` function is the business process controller of this function. It receives an image containing a table as input.

- The most critical operation of this function is **constructing a clear and definite Prompt**. This Prompt directly instructs the large model to perform two tasks: 1. Recognize the table in the image. 2. Return the recognized content in Markdown format. This mandatory requirement for output format is the key to achieving unstructured to structured conversion.
- After constructing the Prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this formatted instruction together.
- After receiving the returned Markdown text from the model interface layer, it performs a file operation: writing this text content to a new `.md` file.
- Finally, it returns structured data containing the new file path and table content.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function continues to serve as the unified "dispatch center". It receives the image and Prompt from `scan_table`, and dispatches the task to the correct backend model implementation based on the current system configuration (`self.llm_platform`).
- Regardless of the backend model, the task of this layer is to handle communication details with specific platforms, ensuring that image and text data are sent correctly, and then returning the pure text returned by the model (here in Markdown format) to the tool layer.

In summary, the general flow of table scanning is: `ToolsManager` receives the image and constructs an instruction "convert the table in this image to Markdown" -> `ToolsManager` calls the model interface -> `model_interface` packages the image and instruction together and sends them to the corresponding model platform based on the configuration -> The model returns text in Markdown format -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` saves the text as a `.md` file and returns the result. This flow demonstrates how the large model's format-following capability can be leveraged to use it as a powerful OCR (Optical Character Recognition) and data structuring tool.

3. Practical Operations

3.1 Configure Online LLM

1. Update the key in the configuration file, open the model interface configuration file

`large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. Fill in your API Key:

Find the corresponding section and paste the API Key you just copied. Here taking Qwen configuration as an example

```
# large_model_interface.yaml

## Qwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose models as needed, such as qwen-
turbo, qwen-plus
```

3. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

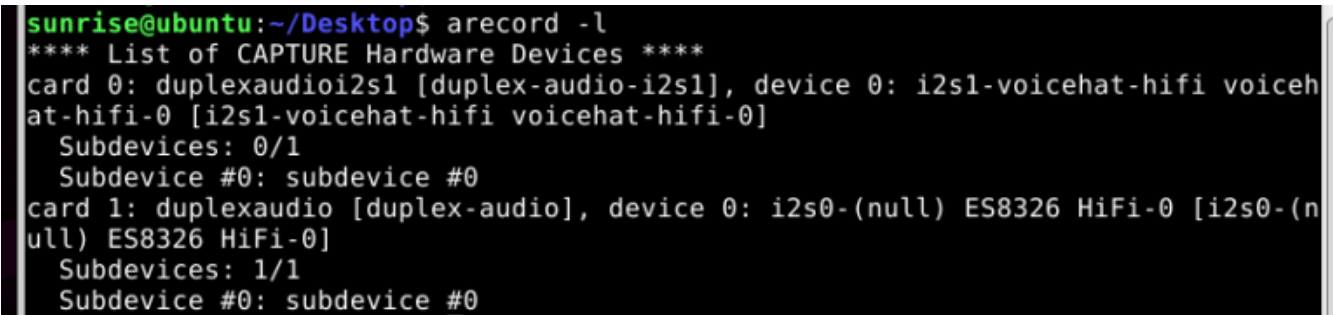
4. Select the online platform to use:

Modify the `llm_platform` parameter to the platform name you want to use

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' # Optional platforms: 'tongyi', 'spark', 'qianfan',
    'openrouter'
```

5. When using the voice module for broadcast, check if a device is occupying the voice module. You can first enter



```
sunrise@ubuntu:~/Desktop$ arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: duplexaudioi2s1 [duplex-audio-i2s1], device 0: i2s1-voicehat-hifi voiceh
at-hifi-0 [i2s1-voicehat-hifi voicehat-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
card 1: duplexaudio [duplex-audio], device 0: i2s0-(null) ES8326 HiFi-0 [i2s0-(n
ull) ES8326 HiFi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

If the output is as shown in the image above, you can directly use the broadcast device number without modification. If different, open the configuration file

```
vim ~/yahboomcar_ws/src/largemodel/largemodel/model_service.py
```

Find the location in the image and modify it to `plughw:1,0` or `hw:1,0`, then save.

```

172     except Exception as e:
173         self.get_logger().error(f"❌ Error in TTS callback: {e}")
174         # 出错时恢复麦克风
175         self.publish_wakeup_signal(False)
176
177     def play_tts_audio(self):
178         """播放TTS音频文件"""
179         try:
180             if not os.path.exists(self.tts_out_path):
181                 self.get_logger().error(f"❌ Audio file not found: {self.tts_out_path}")
182                 return
183
184             # 使用aplay命令播放
185             self.get_logger().info(f"🔊 Playing audio: aplay -D plughw:0,0 {self.tts_out_path}")
186
187             result = subprocess.run(
188                 ["mpg123", "-a", "plughw:1,0", self.tts_out_path],
189                 capture_output=True,
190                 text=False
191             )
192
193             if result.returncode == 0:
194                 self.get_logger().info("✅ TTS playback completed successfully")
195             else:
196                 self.get_logger().error(f"❌ TTS playback failed: {result.stderr}")
197
198         except Exception as e:
199             self.get_logger().error(f"❌ Error playing TTS audio: {e}")
200

```

After modifying the configuration file, you need to recompile in the workspace and source:

```

cd ~/yahboomcar_ws
colcon build && source install/setup.bash

```

3.2 Start and Test the Function

1.Prepare the image file:

Place the image file you want to test in the following path:

```
~/yahboomcar_ws/src/largemodel/resources_file/scan_table
```

Then rename the image to `test_table.jpg`

2.Open a new terminal and start the `audio_io` main program:

Run the following command to enable the microphone and wake-up program:

```
ros2 launch audio_io audio_io_yahboom.launch.py
```

3.Open a new terminal and start the `largemodel` main program:

Run the following command to enable voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

4.Test:

- **Wake-up:** Say to the microphone: "Hello, Digua."

- **Dialogue:** After the speaker responds, you can say: `Analyze the table`
- **Observe logs:** In the terminal running the `launch` file, you should be able to see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, calls the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker, and you should find an MD document saved in the path `~/yahboomcar_ws/src/largemodel/resources_file/scan_table` containing the table information.